LABORATOR 5 – statistica

EX#1 Se aruncă două zaruri corecte cu feţele 1,2,3,4,5,6. Considerăm următoarele evenimente:

(i) A = primul este 1;
(ii) B = al doilea este 6;
(iii) C = suma celor două este 7;
(iv) D = primul este mai mic sau egal cu 2;
(v) E = suma celor două este 7;
(vi) F = al doilea este par.

Creaţi un fişier în Python® prin care să se verifice numeric (frecvenţionist) dacă:

(a) A este independent de C;
(b) B este independent de C;
(c) A, B şi C sunt independente;
(d) D este independent de E;
(e) E este independent de F;
(f) D, E şi F sunt independente.

```
import numpy as np
import matplotlib.pyplot as plt

#%% Ex 1
N = int(1e6)
Z1 = np.random.randint(1,6+1, size = N)
Z2 = np.random.randint(1,6+1, size = N)

A = 1*(Z1 == 1)
B = 1*(Z2 == 6)
C = 1*(Z1+Z2 == 7)

D = 1*(Z1<=2)
```

```
E = 1*(Z1+Z2 == 7)
F = 1*(Z2%2 == 0)


#%% (a) A si C - sunt indespendente
P = np.sum(A*C)/N
prodP = np.sum(A)/N*np.sum(C)/N
print(u'P(A & C):',P)
print(u'P(A)P(C):',prodP)

#%% (b) B si C - sunt independente
P = np.sum(B*C)/N
prodP = np.sum(B)/N * np.sum(C)/N
print('P(B & C):',P)
print('P(B)P(C):',prodP)

#%% (c) A,B si C - nu sunt independente
P = np.sum(A*B*C)/N
prodP = np.sum(A)/N * np.sum(B)/N * np.sum(C)/N
print('P(A & B & C):',P)
print('P(A)P(B)P(C):',prodP)

#%% (d) D si E - sunt independente
P = np.sum(D*E)/N
prodP = np.sum(D)/N * np.sum(E)/N
print('P(D & E):',P)
```

```
print('P(D)P(E):',prodP)

#%% (e) E si F - sunt independente
P = np.sum(E*F)/N
prodP = np.sum(E)/N * np.sum(F)/N
print('P(E & F):',P)
print('P(E)P(F):',prodP)

#%% (e) D, E si F - sunt independente
P = np.sum(D*E*F)/N
prodP = np.sum(D)/N * np.sum(E)/N * np.sum(F)/N
print('P(E & F):',P)
print('P(E)P(F):',prodP)
```

EX#2 Fie X un număr generat aleator uniform în intervalul $[-1, 1]$.

(a) Fie Y un număr generat aleator uniform în intervalul $[-1, 1]$;

(b) Fie $Y = -X$.

Creaţi un fişier în Python® prin care să se verifice numeric (frecvenţionist) dacă X şi Y sunt independente pentru fiecare dintre cazurile (a), respectiv (b).

```
import numpy as np

N = int(1e5)

X = np.random.uniform(-1,1, size = N)
#Y = np.random.uniform(-1,1, size = N) # X si Y sunt independente
Y = -X # X si Y nu sunt independente
```

```
a = -0.1
b = 0.3

c = -0.25
d = 0.5

A = (1*(a<=X)) * (1*(X<=b))
B = (1*(c<=Y)) * (1*(Y<=d))


P = np.sum(A*B) / N
prodP = np.sum(A) / N * np.sum(B) / N
print("P: ", P)
print("prodP: ", prodP)
```

EX#3 Fie $p \in [0,1]$. Creaţi un fişier în Python® prin care să se genereze un număr aleator X distribuit geometric $Geom(p)$

(a) folosind algoritmul de generare din Python®;

(b) $X = \left\lceil \frac{\ln U}{\ln(1-p)} \right\rceil$, unde U este un număr generat aleator uniform în $[0,1]$.

Creaţi un fişier în Python® prin care

(c) să se realizeze N simulări pentru fiecare dintre cazurile (a), respectiv (b);

(d) să se afişeze histrogramele corespunzătoare simulărilor realizate la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

(e) să se afişeze graficul ponderilor $p_k := (1-p)^{k-1}p,\ k = \overline{1,n}$ pentru un $n \in \mathbb{N}$ suficient de mare;

(f) să se estimeze numeric media şi varianţa variabilei aleatoare distribuită geometric $Geom(p)$ folosind simulările de la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

```
#%% Ex 3 - Geom(p)
import numpy as np
import matplotlib.pyplot as plt

p = 0.3
N = 1000000

# (a) Geom(p) - Python
X_a = np.random.geometric(p, size=N)

# (b) Geom(p) - formula inversă
U = np.random.rand(N)
X_b = np.floor(np.log(U) / np.log(1 - p)) + 1
X_b = X_b.astype(int)
```

```
#transformă explicit vectorul în întregi (int)

print('X_a: ', X_a)
print('X_b: ', X_b)

# (c), (d) Histograme
plt.figure()
plt.hist(X_a,bins=50)
#Histogramă = împărţirea datelor în intervale (benzi, „bins”) şi
numărarea câte valori cad în fiecare interval.
#Parametrul bins controlează numărul de intervale.
plt.title('Ex 3 a: N='+str(N))


plt.figure()
plt.hist(X_b, bins=50)
plt.title(f'Ex 3 b: N={N}')


# (e) Grafic ponderi teoretice
n = 50
k = np.arange(1, n + 1)
pk = (1 - p) ** (k - 1) * p
#pk = np.array([(1-p)**(k-1)*p for k in range(1,50)])
#plt.plot(np.arange(1,50), pk,'-o')
plt.figure()
```

```
plt.plot(k, pk, 'o-')
plt.title('Ponderi teoretice Geom(p)')

plt.figure()
plt.semilogy(np.arange(1,51),pk,'-o')
plt.title('Ponderi teoretice Geom(p) - var 2')

# (f) Medii şi varianţe
m_a, v_a = np.mean(X_a), np.var(X_a)
m_b, v_b = np.mean(X_b), np.var(X_b)

print("Geom(p), p =", p)
print("Metoda (a): media =", m_a, ", var =", v_a)
print("Metoda (b): media =", m_b, ", var =", v_b)

print("Valori teoretice: media =", 1/p, ", var =", (1-p)/(p**2))

plt.show()
```

EX#4 Un pacient aşteaptă un donator compatibil. Probabilitatea ca un donator să fie compatibil este $p = 30\%$. Creaţi un fişier în Python® prin care să se determine probabilitatea ca pacientul să fie incompatibil cu cel puţin $k = 5$ donatori (primii k).

#%% Ex 4 – Probabilitatea ca primii k donatori să fie incompatibili

```
p = 0.3 # probabilitatea ca un donator sa fie compatibil
k = 5 # primii k donatori

# Probabilitatea ca toti primii k sa fie incompatibili
P_incompatibili = (1 - p) ** k

print(f"Probabilitatea ca pacientul sa fie incompatibil cu primii {k}
donatori este:", P_incompatibili)

# Parametri
p = 0.3
k = 5

# Probabilitatea ca primii k donatori sa fie incompatibili
prob_incompatibili = (1 - p) ** k

# Probabilitatea ca primul donator sa fie compatibil
prob_primul = p

# Probabilitatea ca al doilea sa fie compatibil dupa ce primul e
incompatibil
prob_al_doilea_dupa_primul = (1 - p) * p

print(f"Probabilitatea ca primii {k} donatori sa fie incompatibili:
{prob_incompatibili:.4f}")
print(f"Probabilitatea ca primul donator sa fie compatibil:
{prob_primul:.2f}")
```

print(f"Probabilitatea ca al doilea sa fie compatibil după ce primul e incompatibil: {prob_al_doilea_dupa_primul:.2f}")

EX#5 Fie $n \in \mathbb{N}$ şi $p \in [0,1]$. Creaţi un fişier în Python® prin care să se genereze un număr aleator X distribuit binomial $Bin(n,p)$

(a) folosind algoritmul de generare din Python®;

(b) simulând aruncarea unui zar măsluit cu feţele $0, 1, 2, \ldots, n$, unde probabilitatea să se obţină faţa $k \in \{0, 1, 2, \ldots, n\}$ este $C_n^k p^k (1-p)^{n-k}$;

(c) $X = \sum_{i=1}^{n} X_i$, unde X_i sunt numere generate aleator şi independent cu distribuţie Bernoulli(p), unde fiecare $X_i \in \{0, 1\}$ este generat simulând aruncarea unei monede măsluite cu probabilitate de succes p.

Creaţi un fişier în Python® prin care

(d) să se realizeze N simulări pentru fiecare dintre cazurile (a), (b), respectiv (c);

(e) să se afişeze histrogramele corespunzătoare simulărilor realizate la (d) (pentru fiecare dintre cazurile (a), (b), respectiv (c));

(f) să se afişeze graficul ponderilor $p_k := C_n^k p^k (1-p)^{n-k}$, $k = \overline{0, n}$;

(g) să se estimeze numeric media şi varianţa variabilei aleatoare distribuită binomial $Bin(n,p)$ folosind simulările de la (d) (pentru fiecare dintre cazurile (a), (b), respectiv (c));

!!!!!!!!!!ex 5

```
#%% Ex 5 - Distributia Binomiala
import numpy as np
import matplotlib.pyplot as plt
import math


# Parametri
n = 3   # numarul de incercari
p = 0.3  # probabilitatea de succes
p1 = 1


# (a) Generare folosind functia din Python
```

```
X_a = np.random.binomial(n, p)
#numărul de succese într-un set de n încercări independente,
#fiecare cu probabilitate p.
print("Metoda (a) - Binomial din Python: X =", X_a)
#Metoda (a) - Binomial din Python: X = 2

# (b) Simulam aruncarea unui 'zar masluit'
valori = np.arange(0, n + 1)
pk = [math.comb(n, k) * (p ** k) * ((1 - p) ** (n - k)) for k in valori]
X_b = np.random.choice(valori, p=pk)
#p=pk → îi spunem funcţiei cu ce probabilitate să aleagă fiecare valoare
print("Metoda (b) - Zar masluit: X =", X_b)
#Metoda (b) - Zar masluit: X = 1

# (c) X = suma de n variabile Bernoulli(p)
Xi = (np.random.rand(n) < p).astype(int)
print(Xi) #[0 1 0 1 0 1 0 0 0 0]
X_c = np.sum(Xi)
print("Metoda (c) - Suma de Bernoulli:", X_c) # Metoda (c) - Suma de
Bernoulli: 3

# Datele problemei
n = 3
p = 0.3
N = 10000

# (a) Generare directă din Python
```

```
X_a = np.random.binomial(n, p, size=N)

# (b) Zar "măsluit" cu distribuţia binomială
valori = np.arange(0, n + 1)
pk = [math.comb(n, k) * (p ** k) * ((1 - p) ** (n - k)) for k in valori]
X_b = np.random.choice(valori, size=N, p=pk)

# (c) Suma a n variabile Bernoulli(p)
X_c = np.zeros(N)
for i in range(N):
    Xi = (np.random.rand(n) < p).astype(int)
    X_c[i] = np.sum(Xi)

# (e) Histograme pentru fiecare caz
plt.figure()
#plt.hist(X_a, bins=np.arange(-0.5, n + 1.5, 1), rwidth=0.8)
plt.hist(X_a, bins = 50)
plt.title("(a) Binomial - Python, N=" + str(N))

plt.figure()
plt.hist(X_b, bins=np.arange(-0.5, n + 1.5, 1), rwidth=0.8)
plt.title("(b) Binomial - Zar măsluit, N=" + str(N))

plt.figure()
plt.hist(X_c, bins=np.arange(-0.5, n + 1.5, 1), rwidth=0.8)
plt.title("(c) Binomial - Suma Bernoulli, N=" + str(N))
```

```
plt.show()


# (f) grafic teoretic
plt.figure()
plt.plot(valori, pk, 'o-', label='p_k')
plt.title("Ponderi teoretice Binomial")
plt.xlabel("k")
plt.ylabel("P(X=k)")
plt.show()


# Media și varianța pentru metoda (a)
m_a = np.mean(X_a)
v_a = np.var(X_a)

# Media și varianța pentru metoda (b)
m_b = np.mean(X_b)
v_b = np.var(X_b)

# Media și varianța pentru metoda (c)
m_c = np.mean(X_c)
v_c = np.var(X_c)
```

```
# Afişare rezultate
print("Estimări numerice pentru Binomial(n={}, p={}):".format(n, p))
print("Metoda (a) - media: {:.4f}, varianţă: {:.4f}".format(m_a, v_a))
print("Metoda (b) - media: {:.4f}, varianţă: {:.4f}".format(m_b, v_b))
print("Metoda (c) - media: {:.4f}, varianţă: {:.4f}".format(m_c, v_c))

# Media şi varianţa teoretică
m_teoretic = n*p
v_teoretic = n*p*(1-p)
print("Valori teoretice - media: {:.4f}, varianţă:
{:.4f}".format(m_teoretic, v_teoretic))
```

EX#6 La o companie se prezintă anual la interviu de angajare un număr de $n = 500$ indivizi. Probabilitatea ca un individ să fie angajat este $p = 3\%$. Creaţi un fişier în Python® prin care să se determine probabilitatea ca într-un an să fie angajaţi cel puţin $k = 10$ indivizi.

Indicaţii Python®: `numpy`, `numpy.random`, `scipy.stats`, `matplotlib.pyplot`, `matplotlib.pyplot.hist`

```
import numpy as np

# Parametri
n = 500
p = 0.03
k = 10
N = 100000  # număr de simulări
```

```
# Generăm N valori din Binomial(n, p)
X = np.random.binomial(n, p, size=N)

# Calculăm proporţia simulărilor în care X >= k
prob_estimata = np.mean(X >= k)

print("Probabilitatea estimată ca cel putin 10 persoane sa fie angajate: {:.4f}".format(prob_estimata))
```